

BSc (Hons) in **Computer Science**

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026

Faculty of Computing

Practical 07

> **Module Objective:** In previous weeks, your Drow Ranger agent used A* Search to find a physical path to the goal. However, it blindly assumed every non-wall tile was safe. Today, we implement a **Knowledge Base (KB)** and a **Forward Chaining Inference Engine**. Your agent must now mathematically prove a cell is logically *Feasible* before taking a step, using real-time rules about map vision and enemy threats.

> **Before you start:** This lab continues the project from Labs 1–4. Open the same project and check that `SearchAgent.astar_search()` in `agent.py` works.
>
> - Parts 1, 2 & 4: new files (`logic_engine.py` , `test_logic.py`) in the project folder.
> - Part 3: edit your existing `agent.py` .
> - **Create a separate branch `lab-07`** and start your work there. When finished, **commit your code and push the branch**, as you did in Labs 1–4.
> - **Upload this worksheet's answered PDF to the submission portal.**

Part 1: The Logic Engine Architecture

Step 1.1: Building the Knowledge Base (KB)

> Instead of hardcoding hundreds of nested `if-else` statements in our agent's movement logic, we will build a declarative Knowledge Base. You need to design a Python class that stores **Facts** (current percepts) and **Rules** (Horn Clauses).

1. Create a new file named `logic_engine.py` .
2. Implement a `KnowledgeBase` class based on the following structural requirements:

```
CLASS KnowledgeBase:
    // Attributes
    DECLARE facts AS Set                    // To store unique string fa
    DECLARE rules AS List                   // To store rules as Tuples

    // Methods
    FUNCTION tell_fact(fact_string):
        ADD fact_string TO facts

    FUNCTION tell_rule(premise_list, conclusion_string):
        APPEND Tuple(premise_list, conclusion_string) TO rules

    FUNCTION clear_facts():
        EMPTY the facts Set
```

Part 2: Implementing Forward Chaining

Step 2.1: The Inference Algorithm

> The agent needs an Inference Engine to deduce new information from its raw sensors. Translate the following Data-Driven **Forward Chaining** algorithm into a Python method inside your `KnowledgeBase` class.

1. Write the `forward_chain()` method.
2. Ensure you use a `while` loop that continues to iterate over the rules until a full pass results in absolutely no new facts being deduced.

```
FUNCTION forward_chain():
    DECLARE new_facts_added = TRUE

    WHILE new_facts_added == TRUE:
        new_facts_added = FALSE

        FOR EACH (premises, conclusion) IN rules:
            IF conclusion IS NOT IN facts:
```

```
// Modus Ponens Check
IF ALL premises ARE IN facts:
    ADD conclusion TO facts
    new_facts_added = TRUE
```

Part 3: Hooking Logic into the Grid Game

Step 3.1: Defining the Game Constraints

> Now we translate the game's safety constraints into our logic engine using Propositional Logic.

1. In `agent.py`, add `from logic_engine import KnowledgeBase`, then create the KB in `SearchAgent.__init__`: `self.kb = KnowledgeBase()`.
2. Use your `tell_rule` method to define the following safety rules:
   Rule 1: `TargetVisible ∧ HasDust ⇒ SafeToEngage`
   Rule 2: `SafeToEngage ∧ BloodseekerMissing ⇒ Retreat`

The game does not produce these facts, so you supply them per tile as a dictionary: `{(x, y): [facts]}`. A tile where `Retreat` is deduced is unsafe to enter.

Step 3.2: Validating Feasibility in A*

> Modify your existing A* pathfinding algorithm to consult the Knowledge Base before expanding a node.

1. Add a parameter `tile_facts=None` to `astar_search()` (treat `None` as `{}`).
2. Locate the neighbor-generation loop in your A* code (where you normally check for physical walls).
3. Before adding a neighbor to the `open_list`, clear the KB facts.
4. Feed that tile's facts into the KB: `for fact in tile_facts.get(successor, []): self.kb.tell_fact(fact)`.
5. Run `self.kb.forward_chain()`.

6. If `'Retreat'` is deduced and exists in `self.kb.facts`, mark the tile as **Infeasible**. Skip it (do not add it to the open list), even if it is physically reachable.

Step 3.3: Run It in the Grid

1. In `sense_and_act()`, define sample facts and pass them to A*:

```
tile_facts = {
    (1, 0): ['TargetVisible', 'HasDust', 'BloodseekerMissing'],  # de
    (0, 1): ['TargetVisible', 'HasDust'],                        # Sa
}
self.plan = self.astar_search(position, nearest_food, walls, grid_si
```

2. Run `visual_grid_game.py`. The agent should never step onto `(1, 0)`.

Part 4: Self-Evaluation Test Cases

4.1 Automated Validation

Create a file named `test_logic.py`. Copy and run the following Python assert statements to verify your Forward Chaining engine works correctly mathematically before you attempt to integrate it into the visual grid.

```
from logic_engine import KnowledgeBase

def test_forward_chaining():
    kb = KnowledgeBase()

    # Add Domain Rules
    kb.tell_rule(['TargetVisible', 'HasDust'], 'SafeToEngage')
    kb.tell_rule(['SafeToEngage', 'BloodseekerMissing'], 'Retreat')

    # Test Case 1: Safe Engagement
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
```

```
        kb.forward_chain()
        assert 'SafeToEngage' in kb.facts, "Test 1 Failed: Should deduce
        assert 'Retreat' not in kb.facts, "Test 1 Failed: Should NOT ded

        # Test Case 2: Unsafe Engagement (Bloodseeker Missing)
        kb.clear_facts()
        kb.tell_fact('TargetVisible')
        kb.tell_fact('HasDust')
        kb.tell_fact('BloodseekerMissing')
        kb.forward_chain()
        assert 'Retreat' in kb.facts, "Test 2 Failed: Should deduce Retr

        print("✅ All Logic Engine Test Cases Passed!")

    if __name__ == "__main__":
        test_forward_chaining()
```

Part 5: Theory-to-Implementation Mapping

> Based on the code you just wrote and the lecture on Logical Reasoning, complete the following analytical questions to explain *how* your code implements the theory.

1. Reachability vs. Feasibility (Apply)

In Step 3.2, you modified the A* algorithm's neighbor-generation loop. How does your new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.

Reachability is checked by _get_neighbours(), which excludes walls and positions outside the grid. Feasibility is checked by clearing the Knowledge Base's facts, loading the neighbor's tile facts, and running forward_chain(). If the rules deduce Retreat, A* skips that neighbor before adding it to the frontier, even though it is physically reachable.

2. Declarative vs. Procedural Paradigms (Analyze)

Why did we build a `KnowledgeBase` class with a `forward_chain()` loop instead of simply writing `if 'TargetVisible' in percepts and 'HasDust'`

`in percepts:` directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?

The KnowledgeBase separates game rules from movement logic. The forward_chain() loop applies all rules and lets conclusions trigger other rules, such as SafeToEngage helping deduce Retreat. Adding 50 new rules increases the rule list and inference work, but the movement code and inference algorithm remain unchanged. Direct conditional checks would make the movement code longer and harder to maintain.

3. Modus Ponens & Horn Clauses (Understand)

The rule `['TargetVisible', 'HasDust'] ⇒ 'SafeToEngage'` is formally known as a Horn Clause. Briefly explain how your Python implementation of the `forward_chain()` `while` loop acts as a mathematical execution of the *Modus Ponens* inference rule.

The all() check verifies that every premise of a rule exists in the facts. If TargetVisible and HasDust are present, SafeToEngage is added. This applies Modus Ponens because true premises imply the rule's conclusion. The while loop repeats until no new facts are added, allowing deduced facts to trigger further rules.

Finalize and Lock Lab 07 Implementation

✅ Lab 05 Theory-to-Implementation Mapping completed!

FACULTY OF COMPUTING